

Introduction to Image Analysis

Rimantas Liulys

Table of contents

1	Assignment Overview	1
2	Code Limitations	2
2.1	C++	2
3	Assignment Details	2
3.1	Image Loading	2
3.1.1	Image Investigations	3
3.1.2	Image loading using <code>libtiff</code>	3
3.1.3	Image combination	4
3.2	Intensity Transformations	4
3.2.1	Power law for intensities (disregarding constant c)	4
3.2.2	Piece-wise linear intensity transformations	4
3.2.3	Histogram calculation for grayscale images	4
3.2.4	Histogram equalization	4
3.2.5	Lookup table intensity transformations	4
3.3	Spatial Filtering and Multistep Image Processing	4
3.3.1	Image classes for pixel intensities	4
3.3.2	Image blurring by spatial filtering	4
3.3.3	Numerical approximations of derivative operators	4
3.3.4	Image sharpening using an intermediate Laplacian calculation	4
3.3.5	Multistep image processing	4
3.4	Image Transformations	4
3.4.1	Calculation of index-to-local and local-to-world transformations	4
3.4.2	Composite affine transformations	4
3.4.3	Nearest Neighbor and Bi-linear Interpolation for visualization of transformed images	4
3.4.4	Visualization of affine transformations	4
3.4.5	QA of AI classification of kidney histology images	5

1 Assignment Overview

Digital image processing is the use of computer algorithms to manipulate, analyze, and process digital images represented as a grid of pixels.

The aim of this project is to get a grasp on the core principles of how digital image processing works in practice, how regular filters, for example, negation, are constructed through code, and gain practical experience in reproducing already existing methods of image manipulation.

2 Code Limitations

This project is built upon a premade base, a collection of files that is usually called a “code skeleton”, which was the scaffolding which allowed us to focus on image analysis and manipulation without the need to also create the interface ourselves.

Since I study programming-adjacent modules, I used C++ as my coding language of choice for this project.

2.1 C++

C++ is a high-performance, general-purpose programming language developed by Bjarne Stroustrup at AT&T Bell Labs in 1979 as an extension of the C programming language. Often described as a “middle-level” language, it bridges the gap by providing the raw speed and direct hardware control of low-level languages alongside the abstraction and structural advantages of high-level languages. Code snippets will be provided as examples over the course of this document.

3 Assignment Details

This assignment is split into three parts. This is the first of the three reports that will be covering this project.

Everything is separated into different parts, which are all separated in this document, shown by different number sectioning.

3.1 Image Loading

To get a better practical understanding of different language formats, their memory layout and how to combine and manipulate this information, tasks 3.1.1 - 3.1.3 were completed.

3.1.1 Image Investigations

```
[zypreaxa@Arch imgset1]$ tiffinfo Kidney1.tif
=== TIFF directory 0 ===
TIFF Directory at offset 0xd3808 (866312)
  Subfile Type: (0 = 0x0)
  Image Width: 1120 Image Length: 2508 Image Depth: 1
  Tile Width: 240 Tile Length: 240
  Bits/Sample: 8
  Compression Scheme: JPEG
  Photometric Interpretation: RGB color
  Samples/Pixel: 3
  Planar Configuration: single image plane
  ImageDescription: Aperio Image Library v12.0.15
18288x40236 [0,100 17927x40136] (240x240) -> 1120x2508 JPEG/RGB Q=92
  JPEG Tables: (574 bytes)
```

Figure 1: Metadata of Kidney1.tif file

3.1.2 Image loading using `libtiff`

3.1.2.1 Loading *tiled* and *striped* formats

3.1.2.1.1 Anomalies encountered

3.1.2.1.2 Anomaly description and reasoning

3.1.2.2 Loading and visualizing sub-images of the `.svs` file

3.1.2.3 Loading subregions of images

3.1.3 Image combination

3.2 Intensity Transformations

3.2.1 Power law for intensities (disregarding constant c)

3.2.2 Piece-wise linear intensity transformations

3.2.3 Histogram calculation for grayscale images

3.2.4 Histogram equalization

3.2.5 Lookup table intensity transformations

3.3 Spatial Filtering and Multistep Image Processing

3.3.1 Image classes for pixel intensities

3.3.1.1 Methods of conversion

3.3.2 Image blurring by spatial filtering

3.3.3 Numerical approximations of derivative operators

3.3.4 Image sharpening using an intermediate Laplacian calculation

3.3.4.1 Benefits

3.3.4.2 Drawbacks

3.3.5 Multistep image processing

3.4 Image Transformations

3.4.1 Calculation of index-to-local and local-to-world transformations

3.4.2 Composite affine transformations

3.4.3 Nearest Neighbor and Bi-linear Interpolation for visualization of transformed images

3.4.4 Visualization of affine transformations

3.4.4.1 Nearest Neighbor type

3.4.4.2 Bi-linear type

3.4.5 QA of AI classification of kidney histology images